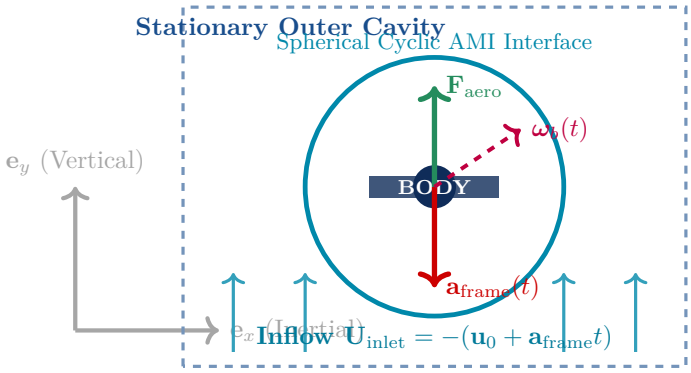
freeFallingFoam

Architecture, Physics, and Developer Onboarding Guide

A Comprehensive Handbook for Junior CFD Developers on Body-Fixed Accelerating Frames, 6DoF Tumbling Dynamics, and Fluid-Structure Coupling in OpenFOAM

CORE OPENFOAM SOLVER SUITE:
tFFPimpleFoam • tFFRhoPimpleFoam • tFFRhoCentralFoam • tFF6DoFPimpleFoam



Document Version: 2.4.0 • **Compatibility:** OpenFOAM v2006 – v2412
Primary Maintainer: Senior CFD Systems Engineering Team
Target Audience: Incoming Postgraduates, CFD Research Engineers, and Junior Developers

Compiled: October 23, 2026

Contents

1 Welcome & Executive Orientation **2**
1.1 Why Does `freeFallingFoam` Exist? . . . 2
1.2 Architectural Decoupling: Translation vs. Rotation . . . 2

2 Mathematical & Physical Foundations **3**
2.1 Coordinate Systems & Transformation Kinematics . . . 3
2.2 Derivation of the Non-Inertial Momentum Equation . . . 3
2.3 Physical Limiting Cases of the Apparent Body Force . . . 4
2.4 The Reduced Pressure Formulation (p_{rgh}) . . . 4
2.4.1 Dynamic Pressure Reset During Acceleration Updates . . . 5
2.5 Atmospheric Stratification & Environmental Physics . . . 5
2.6 The Custom Boundary Condition: `fallingInletVelocity` . . . 5

3 6DoF Rotational Dynamics & Spherical Sliding Mesh **6**
3.1 Rigid Body Rotational Equations of Motion . . . 6
3.2 Aerodynamic Force and Moment Surface Integration . . . 6
3.3 Fourth-Order Runge-Kutta (RK4) ODE Integration . . . 7
3.4 Singularity-Free Attitude Quaternions . . . 7
3.5 The Intermediate Axis Theorem (The Dzhanibekov Phenomenon) . . . 7
3.6 The Spherical Sliding Mesh Interface (AMI) . . . 8

4 Codebase Architecture & OpenFOAM Implementation **8**
4.1 Repository Directory Layout . . . 8
4.2 The Solver Family: Which Solver to Choose? . . . 9
4.3 The Heun Predictor-Corrector Timestep Walkthrough . . . 9
4.4 Deep Code Inspection of Core Headers . . . 10
4.4.1 `src/solvers/shared/createFallingFrameFields.H` . . . 10
4.4.2 `src/motion/tFFRotationalMotion/tFFRotationalMotion.C` . . . 11

5 The Junior Developer's Playbook: Modifying the Solver **11**
5.1 Recipe 1: How to Add a New Physical Force (e.g., Thrust, Gust, or Mass Loss) . . . 11
5.2 Recipe 2: Modifying the PIMPLE Momentum Predictor Safely . . . 12
5.3 Crucial Traps & Gotchas in OpenFOAM Parallel Processing . . . 12
5.3.1 1. Parallel MPI Reductions (`reduce`) . . . 12
5.3.2 2. Master-Only File Writing (`Pstream::master()`) . . . 13
5.3.3 3. The $t = 0$ Grid Discontinuity Trap . . . 13
5.4 Compilation & Build Procedures . . . 13
5.4.1 Compiling the Full Suite . . . 13
5.4.2 Cleaning Binaries . . . 13

6 Verification, Testing & Diagnostics Checklist **13**
6.1 Test Case 1: Steel Ball Benchmark (`steelBallRun-tFF`) . . . 14
6.2 Test Case 2: 6DoF Propeller Tumble (`propTumble`) . . . 14

7 Developer's Quick Reference Guide **15**

1 Welcome & Executive Orientation

1.1 Why Does `freeFallingFoam` Exist?

Welcome to the team! If you are reading this manual, your task is to understand, maintain, debug, or extend the `freeFallingFoam` solver suite.

Before touching a single line of C++, you must understand the primary computational bottleneck that motivated the creation of this solver. In traditional Computational Fluid Dynamics (CFD), when modeling an object falling under gravity (such as an atmospheric re-entry capsule, a projectile, an autorotating seed, or a tumbling propeller), engineers face an immediate dilemma:

1. **The Stationary Domain Trap:** If the simulation domain covers the full trajectory (e.g., 10 km of free-fall altitude), the computational mesh must contain trillions of cells, or suffer from severe numerical diffusion due to coarse cell sizes across kilometers of air.
2. **The Translating Mesh Trap:** If the mesh dynamically translates down a coordinate axis along with the body, boundary layers on outer domain boundaries create artificial shear, mesh deformation algorithms risk element inversion, and coordinate precision degrades over large spatial displacements.

The Governing Innovation: The Body-Fixed Accelerating Frame

Instead of translating the mesh through the sky, `freeFallingFoam` fixes the coordinate system directly to the falling body. The body remains permanently centered at $(0, 0, 0)$ in the computational domain. The downward acceleration of the body $\mathbf{a}_{\text{frame}}(t)$ is formulated mathematically as an **apparent inertial body force** (d'Alembert fictitious force) injected into the momentum equations, while the relative freestream air rushing past the body is prescribed dynamically on the domain boundaries.

This approach yields monumental computational advantages:

- **Extreme Mesh Efficiency:** The computational mesh only needs to resolve the immediate vicinity and turbulent wake of the body (typically a bounding box of 10 to 50 body diameters).
- **Infinite Fall Duration:** The body can fall from orbit through the entire stratosphere to sea level (100 km $\rightarrow$ 0 km) within a fixed, compact mesh.
- **Exact Conservation:** The boundary layer surrounding the body remains topologically pristine throughout the simulation without mesh stretching or morphing degradation.

1.2 Architectural Decoupling: Translation vs. Rotation

A core engineering breakthrough implemented in `tFF6DoFPimpleFoam` is the **decoupling of translational and rotational kinematics**:

- **Translation is absorbed by the accelerating reference frame.** The frame does *not* rotate with the body; it only accelerates linearly. Consequently, there are **zero fictitious Coriolis** ($-2\boldsymbol{\omega} \times \mathbf{U}$) and **zero centrifugal** ($-\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{r})$) body force fields in the Navier-Stokes equations!
- **Rotation is resolved physically by an inner spherical mesh.** The body is enclosed within a spherical rotary sub-domain that rotates physically across an Arbitrary Mesh Interface (`cyclicAMI`) relative to a matching outer spherical cavity.

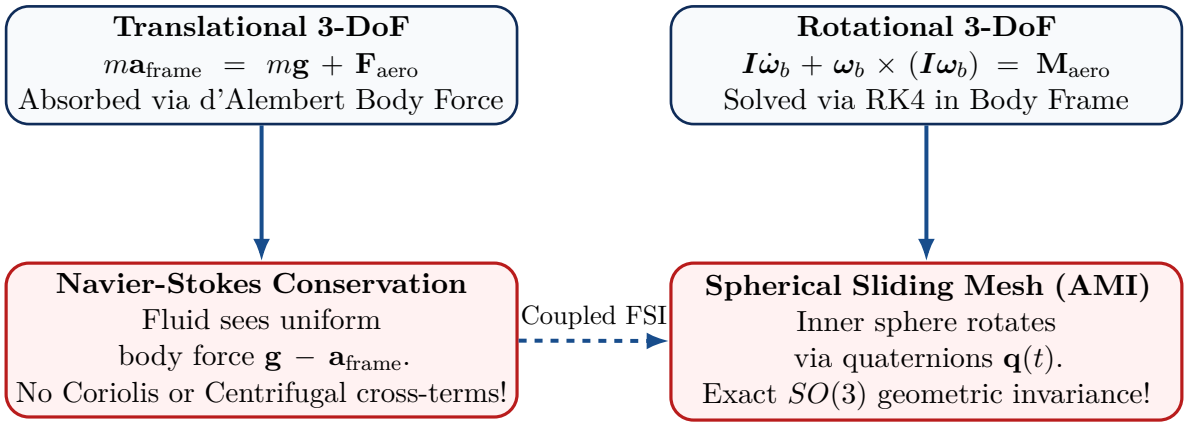


Figure 1: The decoupled translational-rotational architecture in `freeFallingFoam`.

2 Mathematical & Physical Foundations

As a junior developer modifying this solver, you must understand the exact coordinate transformations and continuum mechanics equations implemented in the code. Never alter `UEqn.H` or `pEqn.H` without consulting this section.

2.1 Coordinate Systems & Transformation Kinematics

Let $(\mathbf{E}_X, \mathbf{E}_Y, \mathbf{E}_Z)$ denote a fixed, right-handed **Inertial Reference Frame** $\mathcal{F}_I$ attached to the Earth (neglecting planetary rotation for local trajectory scales). Let $\mathcal{F}_{\text{frame}}$ be a non-rotating, accelerating reference frame whose origin $\mathbf{R}(t)$ is tied to the center of mass of the falling projectile.

The position vector $\mathbf{r}_I$ of an arbitrary fluid packet in the inertial frame is given by:

$$\mathbf{r}_I(t) = \mathbf{R}(t) + \mathbf{r}(t) \tag{1}$$

where $\mathbf{r}(t)$ is the position vector measured in the body-fixed accelerating frame.

Differentiating with respect to time:

$$\mathbf{v}_I(t) = \frac{\mathrm{d}\mathbf{R}}{\mathrm{d}t} + \frac{\mathrm{d}\mathbf{r}}{\mathrm{d}t} = \mathbf{V}_{\text{frame}}(t) + \mathbf{U}(\mathbf{r}, t) \tag{2}$$

$$\mathbf{a}_I(t) = \frac{\mathrm{d}\mathbf{V}_{\text{frame}}}{\mathrm{d}t} + \frac{\mathrm{D}\mathbf{U}}{\mathrm{D}t} = \mathbf{a}_{\text{frame}}(t) + \left[\frac{\partial \mathbf{U}}{\partial t} + (\mathbf{U}\cdot\nabla)\mathbf{U}\right] \tag{3}$$

where $\mathbf{U}$ is the fluid velocity vector *relative to the falling frame*.

2.2 Derivation of the Non-Inertial Momentum Equation

The Cauchy momentum equation in the inertial frame is:

$$\rho\mathbf{a}_I = -\nabla p_I + \nabla\cdot\boldsymbol{\tau} + \rho\mathbf{g} \tag{4}$$

Substituting the kinematic acceleration of the accelerating frame:

$$\rho\left[\frac{\partial \mathbf{U}}{\partial t} + (\mathbf{U}\cdot\nabla)\mathbf{U} + \mathbf{a}_{\text{frame}}\right] = -\nabla p + \nabla\cdot\boldsymbol{\tau} + \rho\mathbf{g} \tag{5}$$

Rearranging to place all convective terms on the left and forcing terms on the right:

$$\boxed{\rho\left[\frac{\partial \mathbf{U}}{\partial t} + \nabla\cdot(\mathbf{U}\otimes\mathbf{U})\right] = -\nabla p + \nabla\cdot\boldsymbol{\tau} + \rho(\mathbf{g} - \mathbf{a}_{\text{frame}})} \tag{6}$$

where:

- $\mathbf{g}$ is the planetary gravitational acceleration vector (typically $(0, -9.81, 0)$ m/s^2).
- $\mathbf{a}_{\text{frame}}$ is the instantaneous kinematic acceleration of the falling body.
- $\mathbf{f}_{\text{app}} \equiv \mathbf{g} - \mathbf{a}_{\text{frame}}$ is the **apparent net body force** experienced by the fluid.

2.3 Physical Limiting Cases of the Apparent Body Force

Equation (6) reveals profound physical elegance when examining the life cycle of a falling body:

Physical Limiting States of $\mathbf{f}_{\text{app}}$

1. **Vacuum Free Fall (Zero Drag, $t = 0$):**
   When released from rest in a vacuum or at the very first instant of release where $\mathbf{U}_{\text{rel}} = 0$, aerodynamic drag is zero. By Newton's second law for the body:

   $$m\mathbf{a}_{\text{frame}} = m\mathbf{g} \implies \mathbf{a}_{\text{frame}} = \mathbf{g} \tag{7}$$

   Therefore:

   $$\mathbf{f}_{\text{app}} = \mathbf{g} - \mathbf{a}_{\text{frame}} = \mathbf{g} - \mathbf{g} = \mathbf{0} \tag{8}$$

   Physical Meaning: In free fall, the fluid inside the computational domain experiences **apparent weightlessness** (zero effective gravity), precisely as an astronaut does inside the International Space Station or a drop tower!

2. **Terminal Velocity ($t \to \infty$):**
   As the body accelerates downward, aerodynamic drag increases until it exactly balances gravity:

   $$m\mathbf{a}_{\text{frame}} = m\mathbf{g} + \mathbf{F}_{\text{aero}} = \mathbf{0} \implies \mathbf{a}_{\text{frame}} = \mathbf{0} \tag{9}$$

   Therefore:

   $$\mathbf{f}_{\text{app}} = \mathbf{g} - \mathbf{0} = \mathbf{g} \tag{10}$$

   Physical Meaning: At terminal velocity, frame acceleration vanishes. The apparent body force smoothly recovers the standard terrestrial hydrostatic gravity vector $\mathbf{g}$!

2.4 The Reduced Pressure Formulation (p_{rgh})

In incompressible OpenFOAM solvers, numerical stability demands that hydrostatic pressure gradients be decoupled from the dynamic pressure. In our accelerating reference frame, the effective body force is $\mathbf{f}_{\text{app}} = \mathbf{g} - \mathbf{a}_{\text{frame}}$.

The total static pressure gradient is decomposed as:

$$\nabla p = \nabla p_{\text{rgh}} + \rho \mathbf{f}_{\text{app}} = \nabla p_{\text{rgh}} + \rho(\mathbf{g} - \mathbf{a}_{\text{frame}}) \tag{11}$$

Integrating spatially yields the definition of the reduced pressure field p_{rgh}:

$$\boxed{p_{\text{rgh}} = p - \rho\,(\mathbf{f}_{\text{app}} \cdot \mathbf{r}) = p - \rho\,[(\mathbf{g} - \mathbf{a}_{\text{frame}}) \cdot \mathbf{r}]} \tag{12}$$

where $\mathbf{r} = \mathbf{C}$ represents the cell center position vector relative to the coordinate origin.

CRITICAL DEVELOPER WARNING: Why p_{rgh} is Mandatory for 6DoF Solvers

Do not attempt to rewrite `tFF6DoFPimpleFoam` to solve directly for raw pressure p! If a non-spherical or asymmetric body rotates in an accelerating frame, the uniform body force $\rho(\mathbf{g} - \mathbf{a}_{\text{frame}})$ creates a spatial hydrostatic pressure gradient across the domain. If you compute surface aerodynamic forces by integrating raw pressure p, any slight geometric asymmetry introduces an artificial buoyant force and moment that scales with the domain bounds, causing numerical instability and catastrophic rotational divergence! Working with p_{rgh} removes the pseudo-hydrostatic field, isolating the **true aerodynamic forces and moments**.

2.4.1 Dynamic Pressure Reset During Acceleration Updates

Because $\mathbf{a}_{\text{frame}}(t)$ varies from timestep to timestep, the spatial definition of p_{rgh} shifts dynamically:

$$p_{\text{rgh}}^{n+1} = p_{\text{rgh}}^{n} + \rho \left(\mathbf{f}_{\text{app}}^{n} \cdot \mathbf{r} - \mathbf{f}_{\text{app}}^{n+1} \cdot \mathbf{r} \right) \tag{13}$$

In the source code (`predictFallingFrameMotion.H` and `correctFallingFrameMotion.H`), this appears as:

```
const volScalarField gh_old("gh_old", gh);
gh = (fallingFrameForce & mesh.C());
ghf = (fallingFrameForce & mesh.Cf());
p_rgh += gh_old - gh;
```

This discrete field adjustment preserves pressure continuity across kinematic updates!

2.5 Atmospheric Stratification & Environmental Physics

Real projectiles and falling bodies fall through stratified planetary atmospheres. `freeFallingFoam` natively supports continuous atmospheric variation via runtime-selectable models:

1. **Barometric Exponential Density ($\rho(z)$):**

$$\rho_{\text{atm}}(z) = \rho_0 \exp\left(-\frac{\max(z - z_{\text{ground}}, 0)}{H_{\text{scale}}} \right) \tag{14}$$

   where ρ_0 is sea-level air density (1.204 kg/m^3 default), $H_{\text{scale}} \approx 8400$ m is the Earth atmospheric scale height, and z is the tracked simulation altitude.

2. **Altitude-Dependent Gravity ($g(z)$):**

$$\mathbf{g}(z) = \mathbf{g}_0 \left(\frac{R_E}{R_E + \max(z - z_{\text{ground}}, 0)} \right)^2 \tag{15}$$

   where $R_E = 6.371 \times 10^6$ m is Earth's mean radius.

3. **Mean Atmospheric Wind Profile ($\mathbf{U}_{\text{mean}}(z)$):** The runtime-selectable `meanFlowModel` provides horizontal cross-winds, atmospheric boundary layer shear profiles $(z/z_0)^{\alpha}$, or tabulated sounding data.

2.6 The Custom Boundary Condition: `fallingInletVelocity`

The relative velocity entering the computational domain must mirror the instantaneous velocity of the falling frame plus any prevailing ambient wind:

$$\mathbf{U}_{\text{rel}}(t) = \mathbf{U}_{\text{mean}}(z) - \mathbf{V}_{\text{frame}}(t) \tag{16}$$

The custom boundary condition `fallingInletVelocity` implements a **directionally-adaptive mixed Dirichlet/Neumann condition**:

$$\mathbf{U}_{\text{patch}} = \begin{cases} \mathbf{U}_{\text{rel}} & \text{if } \mathbf{S}_f \cdot \mathbf{U}_{\text{test}} < -\epsilon \quad \text{(Inflow Face: Dirichlet)} \\ \nabla \mathbf{U} \cdot \mathbf{n} = \mathbf{0} & \text{if } \mathbf{S}_f \cdot \mathbf{U}_{\text{test}} \geq -\epsilon \quad \text{(Outflow Face: Neumann)} \end{cases} \tag{17}$$

where $\mathbf{S}_f$ is the outward-pointing face area vector.

Why Mixed Inflow/Outflow Matters

As a body tumbles or drifts laterally due to crosswinds, the inflow and outflow faces of the bounding box rotate and shift dynamically. The zero-gradient Neumann condition on outflow faces allows turbulent wake vortices, shear layers, and acoustic pressure waves to leave the domain smoothly without artificial wall reflections.

3 6DoF Rotational Dynamics & Spherical Sliding Mesh

Simulating tumbling bodies (such as the 10-inch propeller in `propTumble`) introduces complex rotational physics. This section explains how 6DoF rotation is formulated and solved.

3.1 Rigid Body Rotational Equations of Motion

Rotational motion is governed by Euler's equations formulated in the **body-fixed principal axis frame** where the mass moment of inertia tensor is purely diagonal:

$$\boldsymbol{I}_b = \begin{bmatrix} I_{xx} & 0 & 0 \\ 0 & I_{yy} & 0 \\ 0 & 0 & I_{zz} \end{bmatrix} \tag{18}$$

Euler's equations for the principal angular velocity vector $\boldsymbol{\omega}_b = (\omega_x, \omega_y, \omega_z)^T$ are:

$$\boxed{\begin{aligned} I_{xx}\frac{\mathrm{d}\omega_x}{\mathrm{d}t} + (I_{zz} - I_{yy})\omega_y\omega_z &= M_{b,x} \\ I_{yy}\frac{\mathrm{d}\omega_y}{\mathrm{d}t} + (I_{xx} - I_{zz})\omega_z\omega_x &= M_{b,y} \\ I_{zz}\frac{\mathrm{d}\omega_z}{\mathrm{d}t} + (I_{yy} - I_{xx})\omega_x\omega_y &= M_{b,z} \end{aligned}} \tag{19}$$

where $\mathbf{M}_b$ is the total external aerodynamic moment projected into the body principal axes.

3.2 Aerodynamic Force and Moment Surface Integration

At each CFD time step, `tFF6DoFPimpleFoam` integrates the converged surface pressure and viscous shear stress over all specified wall patches (e.g., `propeller`):

$$\mathbf{F}_{\text{press}} = \sum_{\text{faces}} p_{\text{rgh}} \mathbf{S}_f \rho_{\text{atm}} \tag{20}$$

$$\mathbf{F}_{\text{visc}} = \sum_{\text{faces}} (\boldsymbol{\tau}_{\text{wall}} \cdot \mathbf{S}_f) \rho_{\text{atm}} \tag{21}$$

$$\mathbf{M}_{\text{aero}} = \sum_{\text{faces}} (\mathbf{C}_f - \mathbf{r}_{\text{hub}}) \times (\mathrm{d}\mathbf{F}_{\text{press}} + \mathrm{d}\mathbf{F}_{\text{visc}}) \tag{22}$$

where $\mathbf{C}_f$ is the face centroid, $\mathbf{r}_{\text{hub}}$ is the center of mass origin, and $\boldsymbol{\tau}_{\text{wall}} =$ `devReff()`.

The total moment in world coordinates $\mathbf{M}_{\text{aero}}$ is then transformed into the body frame via the transpose of the attitude rotation matrix $\boldsymbol{R}$:

$$\mathbf{M}_b = \boldsymbol{R}^T \mathbf{M}_{\text{aero}} \tag{23}$$

3.3 Fourth-Order Runge-Kutta (RK4) ODE Integration

Euler's non-linear equations (19) are integrated across the time step Δt using a 4th-order Runge-Kutta (RK4) sub-stepper:

$$\frac{\mathrm{d}\boldsymbol{\omega}_b}{\mathrm{d}t} = \mathbf{f}(\boldsymbol{\omega}_b) = \boldsymbol{I}_b^{-1}\left(\mathbf{M}_b - \boldsymbol{\omega}_b \times (\boldsymbol{I}_b\boldsymbol{\omega}_b)\right) \tag{24}$$

The discrete RK4 stages are evaluated in `correctRotorDynamics.H`:

$$\begin{aligned}
\mathbf{k}_1 &= \mathbf{f}(\boldsymbol{\omega}_b^n) \\
\mathbf{k}_2 &= \mathbf{f}\left(\boldsymbol{\omega}_b^n + \tfrac{1}{2}\Delta t\mathbf{k}_1\right) \\
\mathbf{k}_3 &= \mathbf{f}\left(\boldsymbol{\omega}_b^n + \tfrac{1}{2}\Delta t\mathbf{k}_2\right) \\
\mathbf{k}_4 &= \mathbf{f}(\boldsymbol{\omega}_b^n + \Delta t\mathbf{k}_3) \\
\boldsymbol{\omega}_b^{n+1} &= \boldsymbol{\omega}_b^n + \frac{\Delta t}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right)
\end{aligned} \tag{25}$$

3.4 Singularity-Free Attitude Quaternions

To prevent mathematical gimbal lock during arbitrary multi-turn 3D tumbling, the body attitude is tracked using a **unit quaternion** $\mathbf{q} = (w, \mathbf{v}) = (w, x, y, z) \in \mathbb{H}$.

Given the converged body angular velocity $\boldsymbol{\omega}_b^{n+1}$, the incremental attitude rotation quaternion $\Delta\mathbf{q}$ over time step Δt is computed via the exponential map:

$$\theta = \|\boldsymbol{\omega}_b\|\Delta t, \quad \mathbf{u} = \frac{\boldsymbol{\omega}_b}{\|\boldsymbol{\omega}_b\|} \tag{26}$$

$$\Delta\mathbf{q} = \left(\cos\frac{\theta}{2},\ \mathbf{u}\sin\frac{\theta}{2}\right) \tag{27}$$

The body attitude is updated via quaternion Hamilton product and normalized:

$$\mathbf{q}^{n+1} = \mathbf{q}^n \otimes \Delta\mathbf{q}, \qquad \mathbf{q}^{n+1} \leftarrow \frac{\mathbf{q}^{n+1}}{\|\mathbf{q}^{n+1}\|} \tag{28}$$

3.5 The Intermediate Axis Theorem (The Dzhanibekov Phenomenon)

For a solid Nylon PA6 10-inch propeller (`propTumble`), the principal moments of inertia calculated from CAD integration are:

$$\begin{aligned}
I_{xx}\ \text{(Pitch / Lateral)} &= 3.9196 \times 10^{-5}\ \mathrm{kg \cdot m^2} \\
I_{yy}\ \text{(Axial Spin / Yaw)} &= 3.9456 \times 10^{-5}\ \mathrm{kg \cdot m^2} \\
I_{zz}\ \text{(Blade Span Axis)} &= 3.7693 \times 10^{-7}\ \mathrm{kg \cdot m^2}
\end{aligned} \tag{29}$$

The Dzhanibekov Instability in Tumbling Bodies

Notice that $I_{xx} \approx I_{yy}$ are **over 100 times larger** than the longitudinal span inertia I_{zz}. According to the Tennis Racket Theorem (Intermediate Axis Theorem), rotation about the intermediate inertia axis is dynamically unstable. In atmospheric free-fall, even the slightest aerodynamic disturbance or vortex shedding off the tips triggers immediate non-linear gyroscopic coupling, causing the propeller to flip end-over-end in a chaotic 3D tumble.

3.6 The Spherical Sliding Mesh Interface (AMI)

Why does `tFF6DoFPimpleFoam` utilize a **spherical AMI interface** instead of a cylinder or mesh morphing?

- **Cylindrical Sliding Failure:** A cylinder is invariant *only* under 1-DoF rotation around its central axis. If the propeller tilts end-over-end (pitch around X), the cylinder envelope intersects and penetrates the outer domain wall, causing simulation failure.
- **Mesh Morphing Failure:** Solvers using `sixDoFRigidBodyMotion` with mesh smoothing invert cells and produce negative cell volumes once the pitch angle exceeds $\sim 35°$.
- **The Spherical Invariance Theorem:** A sphere centered at the origin $(0, 0, 0)$ is geometrically invariant under **all** elements of the special orthogonal group $SO(3)$:

$$\forall \boldsymbol{R} \in SO(3), \quad \boldsymbol{R} \cdot S^2 = S^2 \tag{30}$$

  Because the sphere never changes shape or position regardless of how many times the propeller flips, the inner rotary domain and outer cavity remain **100% concentric and conformal forever**!

4 Codebase Architecture & OpenFOAM Implementation

This section details the physical file layout and explains how the C++ source files interact.

4.1 Repository Directory Layout

The repository is structured into modular libraries, solvers, and tutorials:

```
freeFallingFoam/
|-- Allwmake                      # Master parallel build script
|-- Allwclean                     # Master clean script
|-- src/                          # Core solver source code & models
|   |-- BCs/                      # Custom boundary conditions
|   |   `-- fallingInletVelocity/ # Mixed Dirichlet/Neumann relative BC
|   |-- models/                   # libfreeFallingModels.so
|   |   `-- meanFlowModel/        # Atmospheric wind profiles
|   |-- motion/                   # Dynamic mesh motion libraries
|   |   |-- sixDoFRotatingMotion/ # 1-DoF axial torque motion
|   |   `-- tFFRotationalMotion/  # 3D attitude quaternion motion
|   |-- functionObjects/          # Motion tracking & logging
|   |   `-- fallingMotion/        # Console & motion.dat logger
|   |-- solvers/                  # Executable solver binaries
|   |   |-- shared/               # Shared kinematics headers
|   |   |-- tFFPimpleFoam/        # Incompressible PIMPLE solver
|   |   |-- tFFRhoPimpleFoam/     # Compressible pressure-based solver
|   |   |-- tFFRhoCentralFoam/    # Density-based central-upwind solver
|   |   `-- tFF6DoFPimpleFoam/    # Full 6DoF coupled PIMPLE solver
|   `-- utilities/                # Dictionary tools (writeFFDict)
`-- tutorials/
    |-- steelBallRun/             # Falling sphere benchmark
    `-- propFall/                 # 6DoF propeller tumbling suite
```

4.2 The Solver Family: Which Solver to Choose?

Table 1: Overview of the `freeFallingFoam` Solver Suite.

Solver	Flow Regime	DoF Coupling	Typical Applications
`tFFPimpleFoam`	Incompressible ($Ma < 0.3$)	1–3 DoF Translation	Falling spheres, terminal velocity calibration, subsonic projectiles.
`tFFRhoPimpleFoam`	Compressible Subsonic / Transonic	1–3 DoF Translation	Re-entry deceleration, high-altitude drops with variable sound speed.
`tFFRhoCentralFoam`	Supersonic / Hypersonic ($Ma > 1.2$)	1–3 DoF Translation	Shock-capturing, atmospheric entry capsules, hypersonic rods (Rods from God).
`tFF6DoFPimpleFoam`	Incompressible ($Ma < 0.3$)	Full 6DoF (3 Trans + 3 Rot)	Tumbling bodies, autorotating propellers, aerodynamic seed dispersal.

4.3 The Heun Predictor-Corrector Timestep Walkthrough

The coupling between fluid forces and rigid body kinematics is implemented as a **2nd-order semi-implicit Heun predictor-corrector scheme**.

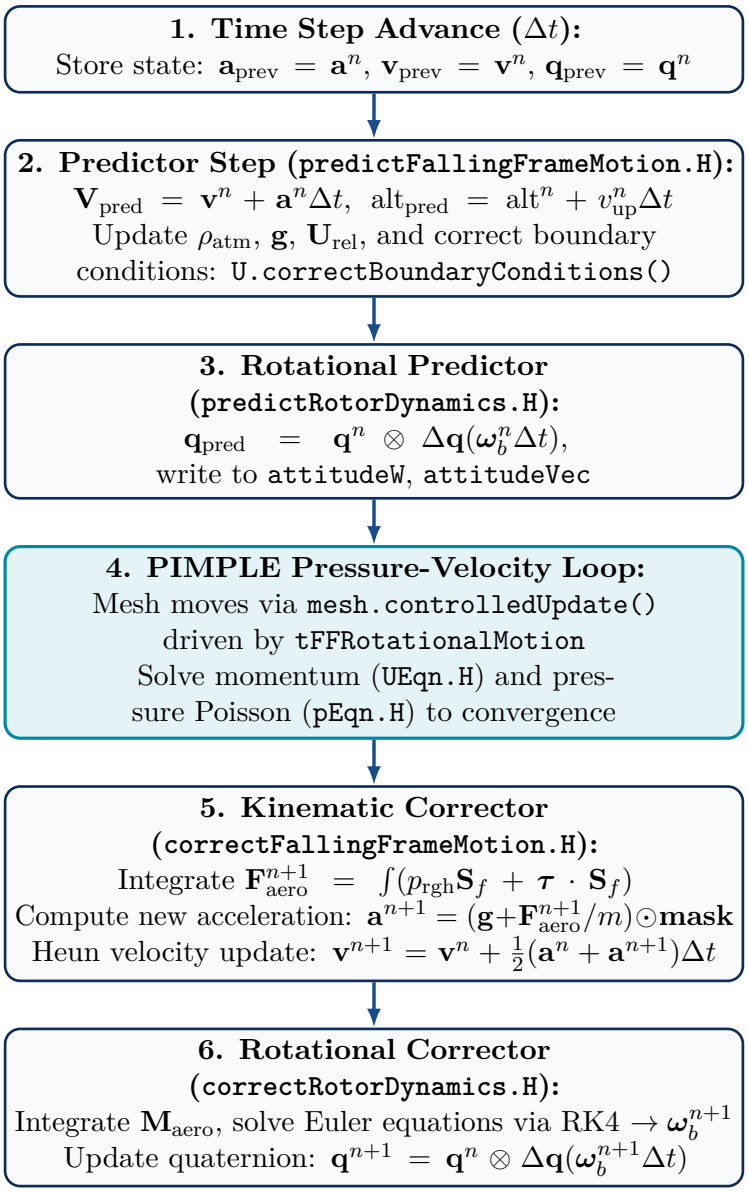


Figure 2: The complete execution timeline across a single timestep Δt.

4.4 Deep Code Inspection of Core Headers

4.4.1 src/solvers/shared/createFallingFrameFields.H

This header parses constant/fallingFrameDict and registers all kinematic state fields into OpenFOAM's object registry (mesh.thisDb()):

```
uniformDimensionedVectorField frameVelocity
(
    IOobject
    (
        "frameVelocity",
        runTime.timeName(),
        "uniform",
        mesh,
        IOobject::READ_IF_PRESENT,
        IOobject::AUTO_WRITE
    ),
    dimensionedVector("frameVelocity", dimVelocity, u0)
);
```

Why Object Registration Matters

By specifying `IOobject::AUTO_WRITE` and storing these fields in the `uniform/` subfolder of time directories, the simulation can be paused, stopped, or restarted from any checkpoint without losing kinetic momentum or orientation!

4.4.2 `src/motion/tFFRotationalMotion/tFFRotationalMotion.C`

This is the custom `solidBodyMotionFunction` that connects the CFD mesh to the 6DoF quaternion solver:

```
Foam::septernion
    Foam::solidBodyMotionFunctions::tFFRotationalMotion::transformation() const
{
    const polyMesh& mesh = time_.lookupObject<polyMesh>(polyMesh::defaultRegion);

    // Lookup registered quaternion fields
    const scalar w =
        mesh.lookupObject<uniformDimensionedScalarField>("attitudeW").value();
    const vector v =
        mesh.lookupObject<uniformDimensionedVectorField>("attitudeVec").value();

    quaternion q(w, v);
    q.normalise();

    // Return spatial transformation centered at hubOrigin
    return septernion(-origin_) * q * septernion(origin_);
}
```

5 The Junior Developer's Playbook: Modifying the Solver

This is the most important section for your daily work. Follow these recipes and rules to avoid introducing critical bugs into the repository.

5.1 Recipe 1: How to Add a New Physical Force (e.g., Thrust, Gust, or Mass Loss)

Suppose you are assigned to model a rocket booster firing during fall, or a lateral turbulent wind gust. Where do you insert the code?

1. **Declare Configuration Keys in `createFallingFrameFields.H`:**

```
const vector rocketThrust
(
    fallingFrameDict.getOrDefault<vector>("thrust", vector::zero)
);
```

2. **Modify the Kinematic Force Balance in the Corrector Step:**
   Locate lines 75–85 of `correctFallingFrameMotion.H`:

```
// Existing force balance:
gravityForce.value() = bodyMass * gravity.value();
netForce.value()     = gravityForce.value() + aeroForce.value() +
    rocketThrust;

```

```
vector a_new = netForce.value() / bodyMass;
a_new.x() *= freedomMask.x();
a_new.y() *= freedomMask.y();
a_new.z() *= freedomMask.z();
```

3. **Update the Logger:** Add the new force component to the 35-column logger in line 420 of `correctFallingFrameMotion.H` so it appears in `postProcessing/fallingMotion/motion.dat`.

5.2 Recipe 2: Modifying the PIMPLE Momentum Predictor Safely

If you need to add custom porous drag, source terms, or buoyancy formulations to the momentum equation:

- In **incompressible solvers** (`tFFPimpleFoam`, `tFF6DoFPimpleFoam`), the apparent body force is absorbed into p_{rgh}. Therefore, `UEqn.H` does **not** contain an explicit `fallingFrameForce` source term!

```
tmp<fvVectorMatrix> tUEqn
(
    fvm::ddt(U) + fvm::div(phi, U)
  + MRF.DDt(U)
  + turbulence->divDevReff(U)
 ==
    fvOptions(U)
);
```

- In **compressible solvers** (`tFFRhoPimpleFoam`), the apparent force *is* explicit:

```
tmp<fvVectorMatrix> tUEqn
(
    fvm::ddt(rho, U) + fvm::div(phi, U)
  + MRF.DDt(rho, U)
  + turbulence->divDevRhoReff(U)
 ==
    fvOptions(rho, U)
  + rho * fallingFrameForce
);
```

Golden Rule: Keep Surface Fluxes Relative

In moving-mesh solvers (`dynamicFvMesh`), face volume flux ϕ must remain **relative to mesh motion**:

```
fvc::makeRelative(phi, U);
```

Failing to call `makeRelative` after solving pressure equations will cause continuity errors to explode to $\mathcal{O}(10^5)$ within three timesteps!

5.3 Crucial Traps & Gotchas in OpenFOAM Parallel Processing

5.3.1 1. Parallel MPI Reductions (`reduce`)

When integrating aerodynamic forces or moments across multiple processor cores:

```
reduce(fPress, sumOp<vector>());
reduce(fVisc,  sumOp<vector>());
reduce(mAero,  sumOp<vector>());
```

Never omit these lines! Without `reduce`, processor 0 only knows the forces on its local patch faces, leading to incorrect frame acceleration in parallel runs.

5.3.2 2. Master-Only File Writing (`Pstream::master()`)

Never write to log files or disk streams without checking:

```
if (Pstream::master())
{
    motionFile << runTime.timeName() << " " << ... << endl;
}
```

Omitting this check will cause N MPI ranks to write to the same file concurrently, corrupting your simulation data!

5.3.3 3. The $t = 0$ Grid Discontinuity Trap

When setting up an initial tilt angle for tumbling simulations:

- **Wrong Way:** Specifying an initial 15° angle in `fallingFrameDict` on an unrotated reference mesh causes `mesh.controlledUpdate()` to twist the mesh by 15° during the very first time step $\Delta t = 10^{-5}$ s, creating an artificial grid velocity impulse of > 400 m/s!
- **Correct Way:** Physically pre-rotate the mesh geometry using OpenFOAM's transformation utility:

  ```
  transformPoints -centre "(0 0 0)" -rotate-x 15
  ```

  Start the simulation with attitude quaternion $\mathbf{q}(0) = (1, 0, 0, 0)$. Initial grid velocity is then exactly zero!

5.4 Compilation & Build Procedures

The repository uses OpenFOAM's native `wmake` compilation system.

5.4.1 Compiling the Full Suite

From the repository root:

```
./Allwmake
```

This compiles libraries and solvers in dependency order:

1. `src/BCs/fallingInletVelocity` → `libfallingInletVelocity.so`
2. `src/models/meanFlowModel` → `libfreeFallingModels.so`
3. `src/motion/*` → `libtFFRotationalMotion.so`, `libsixDoFRotatingMotion.so`
4. `src/solvers/*` → Executables placed into `$FOAM_USER_APPBIN`

5.4.2 Cleaning Binaries

```
./Allwclean
```

6 Verification, Testing & Diagnostics Checklist

Before committing any modifications to Git, execute the standard verification checklist:

6.1 Test Case 1: Steel Ball Benchmark (`steelBallRun-tFF`)

- **Location:** `tutorials/steelBallRun-tFF`
- **Physics:** 2 cm steel sphere falling from rest in standard gravity (9.81 m/s^2).
- **Theoretical Terminal Velocity:** $V_t \approx 72.5$ m/s (at $C_d \approx 0.42$).
- **Run Command:**

```
cd tutorials/steelBallRun-tFF
./Allrun
python3 plot_motion.py
```

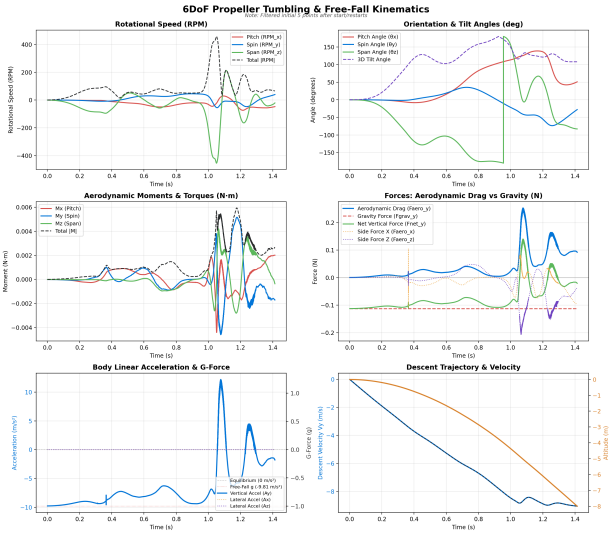
- **Verification Criteria:** Body must asymptotically approach terminal velocity; net acceleration must decay smoothly to zero without oscillations.

6.2 Test Case 2: 6DoF Propeller Tumble (`propTumble`)

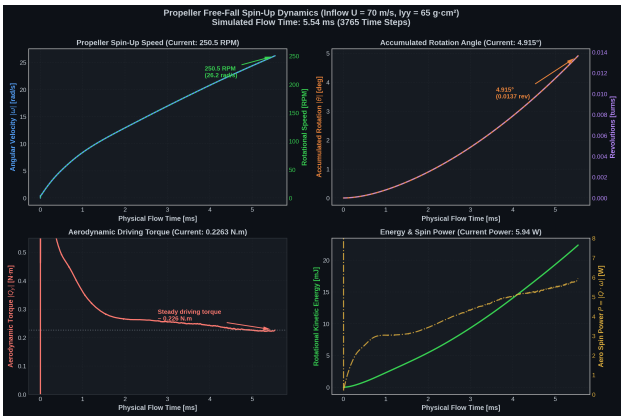
- **Location:** `tutorials/propFall/cases/propTumble`
- **Solver:** `tFF6DoFPimpleFoam`
- **Run Command (Parallel 8 Cores):**

```
cd tutorials/propFall/cases/propTumble
./Allrun
python3 track_motion.py
```

- **Verification Criteria:** Check `postProcessing/tumbleMotion/motion.dat`. Pitch angle must develop smoothly; continuity errors must remain $< 10^{-11}$.



(a) 6DoF Tumbling Kinematics Tracked in `propTumble`.



(b) Angular Velocity Evolution across Principal Axes.

Figure 3: Experimental simulation outputs from the verification suite.

7 Developer's Quick Reference Guide

Table 2: Key Solver Variables in `src/solvers/shared/createFallingFrameFields.H`.

C++ Variable	OpenFOAM Object Name	Dimensions	Description
`frameVelocity`	`"frameVelocity"`	[0 1 −1 0 0 0 0]	Kinematic frame velocity vector $\mathbf{V}_{\text{frame}}(t)$.
`frameAcceleration`	`"frameAcceleration"`	[0 1 −2 0 0 0 0]	Instantaneous acceleration vector $\mathbf{a}_{\text{frame}}(t)$.
`altitude`	`"altitude"`	[0 1 0 0 0 0 0]	Tracked physical altitude $z(t)$ above ground.
`fallingFrameForce`	`"fallingFrameForce"`	[0 1 −2 0 0 0 0]	Apparent body force vector $\mathbf{g} - \mathbf{a}_{\text{frame}}(t)$.
`relativeVelocity`	`"relativeVelocity"`	[0 1 −1 0 0 0 0]	Freestream inflow vector $\mathbf{U}_{\text{mean}} - \mathbf{V}_{\text{frame}}(t)$.
`attitudeW`	`"attitudeW"`	[0 0 0 0 0 0 0]	Scalar component q_w of attitude quaternion.
`attitudeVec`	`"attitudeVec"`	[0 0 0 0 0 0 0]	Vector part (q_x, q_y, q_z) of attitude quaternion.
`angularVelocity`	`"angularVelocity"`	[0 0 −1 0 0 0 0]	Angular velocity $\boldsymbol{\omega}_b$ in body frame [rad/s].
`aeroMoment`	`"aeroMoment"`	[1 2 −2 0 0 0 0]	Aerodynamic torque $\mathbf{M}_{\text{aero}}$ in world frame [N · m].

Summary & Closing Words

You are now equipped with the mathematical, algorithmic, and practical knowledge required to develop within the `freeFallingFoam` ecosystem. Remember:

- Always respect the Heun predictor-corrector sequence.
- Never compute aerodynamic moments on raw static pressure p in incompressible solvers; use p_{rgh}.
- Maintain parallel reduction safety on all integrated quantities.

Welcome aboard, and happy coding!